Name : Bhoomi Mangesh Naik
Class : D15C
Roll No. : 34
Practical No. : 10

Aim : Explore and implement Autoencoders for image denoising

Theory :

1. Dataset Source

The dataset used for this experiment is obtained from Kaggle:
Fashion MNIST Dataset
https://www.kaggle.com/datasets/zalando-research/fashionmnist

2. Dataset Description

The Fashion MNIST dataset consists of grayscale images of clothing items used for image processing and computer vision tasks. In this experiment, the dataset is used for image denoising using autoencoders.

Dataset Characteristics

- Total Records: 70,000 images (60,000 training + 10,000 testing)
- Image Size: 28 × 28 pixels
- Type: Image dataset (grayscale)
- Learning Type: Unsupervised learning

Features Used

Feature	Description
Image Pixels	28×28 grayscale pixel values
Noisy Image	Image with added noise
Clean Image	Original image used as target

3. Mathematical Formulation

3.1 Autoencoder

An autoencoder is a neural network used to learn efficient representations of data by compressing and reconstructing input.

Encoder:
$h = f(Wx + b)$

Decoder:
$\hat{x} = g(W'h + b')$

Where:
x = input image
h = latent representation
$\hat{x}$ = reconstructed image

3.2 Noise Addition

Noise is added to input images to train the model for denoising:

$x_noisy = x + noise$

3.3 Reconstruction Loss

The objective is to minimize the difference between original and reconstructed image:

$Loss = ||x - \hat{x}||^2$

3.4 Activation Function

Common activation functions used:

- ReLU (hidden layers)
- Sigmoid (output layer for normalized images)

3.5 Backpropagation Through Time (BPTT)

Weights are updated using gradient descent:

$$\boxed{W = W - \eta \frac{\partial L}{\partial W}}$$

Where:

- η = learning rate

- $\boxed{\frac{\partial L}{\partial W}}$ = gradient of loss

4. Algorithm Limitations

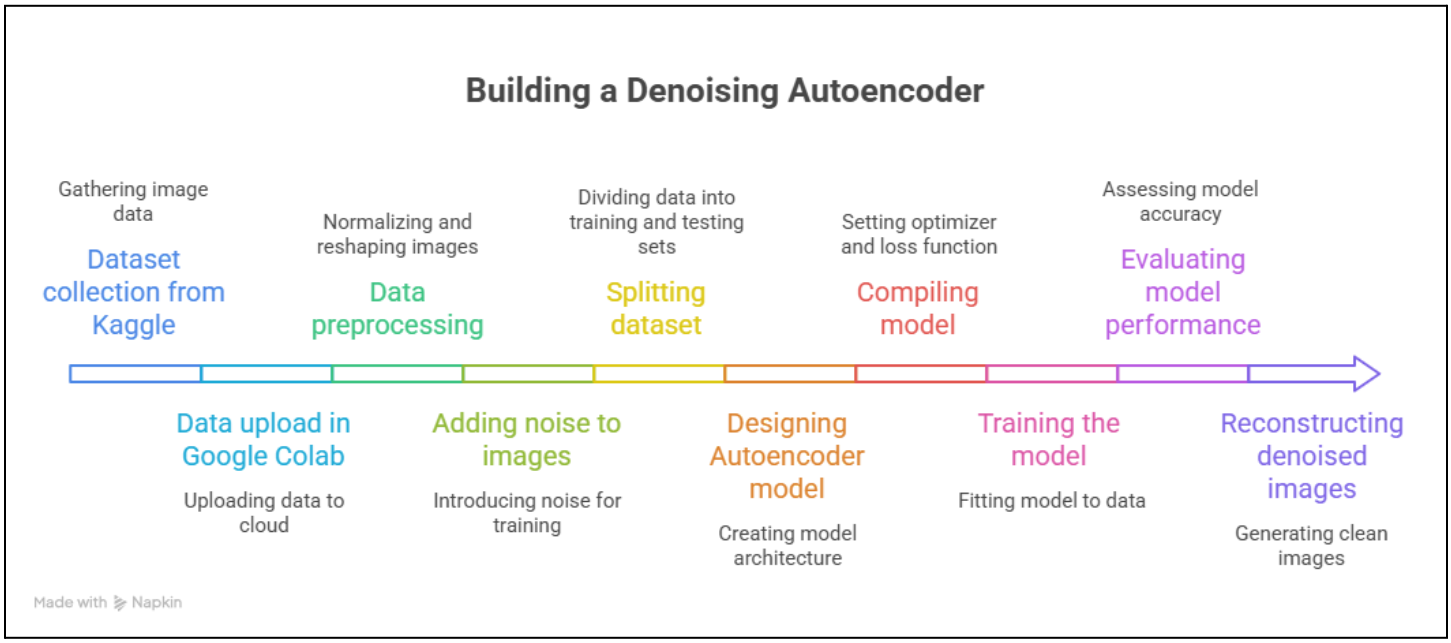
Autoencoder Limitations

- May learn identity function instead of meaningful features
- Requires large dataset for better generalization
- Sensitive to noise level selection
- High computational cost for deep architectures
- Blurry image reconstruction in some cases
- Overfitting without regularization
- Difficult to interpret latent features

5. Methodology / Workflow

- Dataset collection from Kaggle
- Data upload in Google Colab
- Data preprocessing:
    - Normalization of pixel values (0–1)
    - Reshaping images
- Adding noise to images
- Splitting dataset into training and testing sets
- Designing Autoencoder model using Keras/TensorFlow:
    - Encoder layers
    - Bottleneck layer
    - Decoder layers
- Compiling model (optimizer, loss function)
- Training the model (fit function)
- Evaluating model performance
- Reconstructing denoised images

Workflow Diagram:



6. Performance Analysis

- Evaluated using loss (Mean Squared Error)
- Lower loss indicates better reconstruction quality
- Visual comparison of noisy vs denoised images
- Training vs validation loss helps detect overfitting
- Reconstruction quality determines model performance
- Effective denoising indicates successful feature learning

7. Hyperparameter Tuning

Autoencoder Hyperparameters

- Number of encoder/decoder layers
- Number of filters
- Kernel size
- Latent space size
- Learning rate
- Batch size
- Number of epochs
- Activation functions
- Dropout rate

Impact:

- Improves reconstruction quality
- Enhances feature extraction

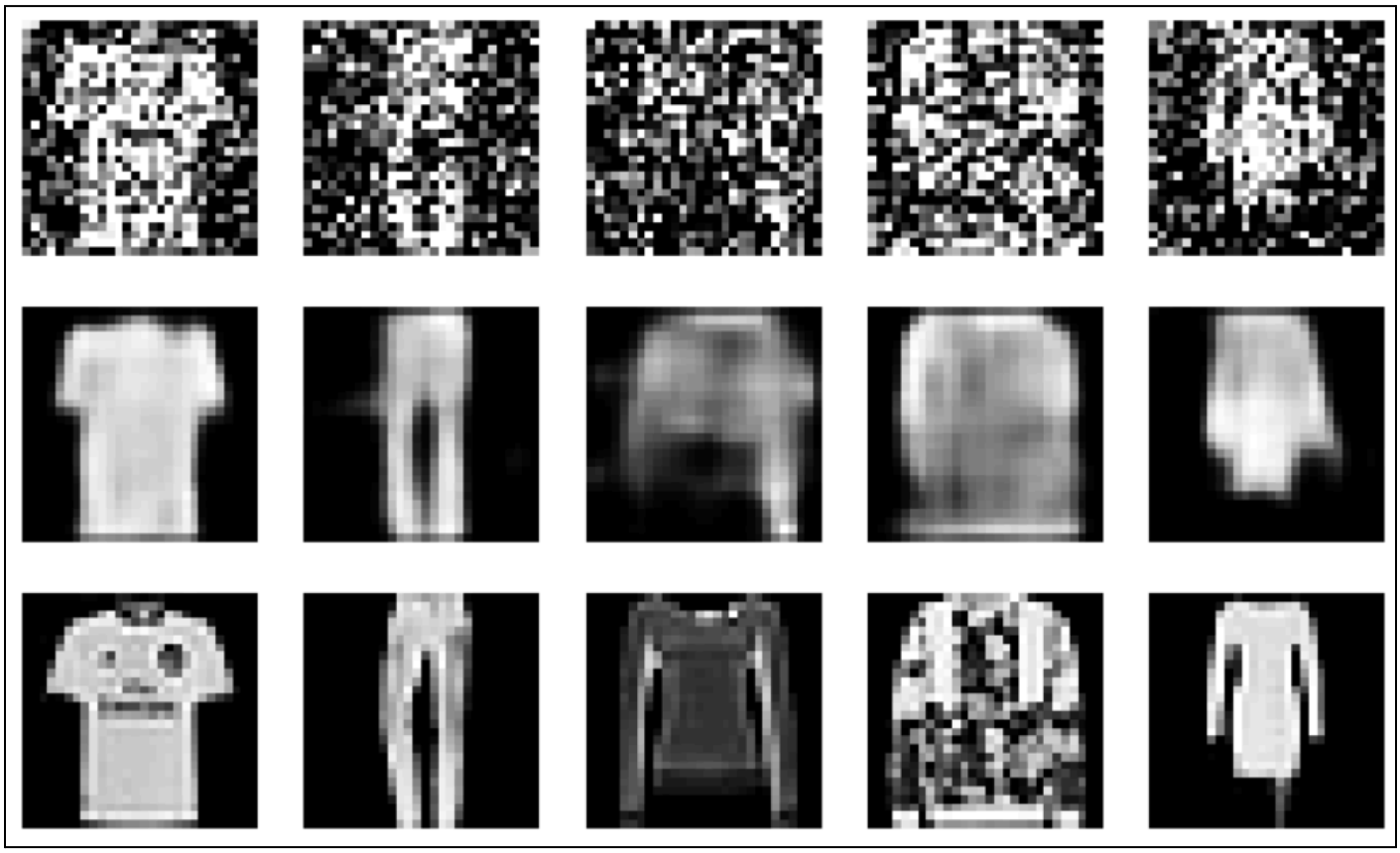
- Reduces noise effectively
- Prevents overfitting
- Improves generalization ability

Code and Output :

```
from google.colab import files
uploaded = files.upload()
import pandas as pd

train_df = pd.read_csv('fashion-mnist_train.csv')
test_df = pd.read_csv('fashion-mnist_test.csv')

train_df.head()
# Drop label column
x_train = train_df.drop('label', axis=1).values
x_test = test_df.drop('label', axis=1).values

# Labels not needed for autoencoder
import numpy as np

x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Reshape into 28x28 images
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
import matplotlib.pyplot as plt

n = 5
plt.figure(figsize=(10,4))

for i in range(n):
    plt.subplot(1, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
```

```
    plt.axis('off')

plt.show()
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D

input_img = Input(shape=(28,28,1))

# Encoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)

x = Conv2D(16, (3,3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2,2), padding='same')(x)

# Decoder
x = Conv2D(16, (3,3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2,2))(x)

x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = UpSampling2D((2,2))(x)

decoded = Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,
    batch_size=128,
    validation_data=(x_test_noisy, x_test)
)
decoded_imgs = autoencoder.predict(x_test_noisy)

n = 5
plt.figure(figsize=(10,6))

for i in range(n):
    # Noisy
    plt.subplot(3, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
```

```
    plt.axis('off')

    # Denoised
    plt.subplot(3, n, i+n+1)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.axis('off')

    # Original
    plt.subplot(3, n, i+2*n+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.axis('off')
plt.show()
```



Google Colab Link for Code and Output : [Link for Code and Output]

Conclusion :

The autoencoder model was successfully implemented for image denoising using the Fashion MNIST dataset. The model effectively learned to reconstruct clean images from noisy inputs, demonstrating its capability in feature extraction. The reconstruction loss decreased over training, indicating good model convergence. Visual results showed significant noise reduction while preserving important image details. Thus, autoencoders prove to be a powerful technique for image denoising tasks.